

Chap7-Exercises_LuisCorreia-745724_v2

October 12, 2025

1 MAP5935 - Statistical Learning (Chapter 7 - Moving Beyond Linearity)

Prof. Christian Jäkel

<https://www.statlearning.com/>

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pyplot import subplots

import statsmodels.api as sm
from statsmodels.gam.api import BSplines, GLMGam

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer, make_column_selector as selector
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.linear_model import RidgeCV
from sklearn.linear_model import LassoCV
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression

from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, GridSearchCV
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

1.1 *Conceptual Exercises*

1.1.1 (3) Suppose we fit a curve with basis functions $b_1(X) = X$, $b_2(X) = (X-1)^2 I_{(X \geq 1)}$. (Note that $I_{(X \geq 1)}$ equals 1 for $X \geq 1$ and 0 otherwise.) We fit the linear regression model.

1.1.2

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon$$

1.1.3 and obtain coefficient estimates $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = -2$. Sketch the estimated curve between $X = -2$ and $X = 2$. Note the intercepts, slopes, and other relevant information.

Solution: We fit a model with basis functions $b_1(X) = X$, $b_2(X) = (X - 1)^2 \mathbb{1}\{X \geq 1\}$.

Given $\hat{\beta}_0 = 1$, $\hat{\beta}_1 = 1$, $\hat{\beta}_2 = -2$, the fitted function is

$$\widehat{Y}(x) = 1 + x - 2(x - 1)^2 \mathbb{1}\{x \geq 1\}.$$

Piecewise:

$$\widehat{Y}(x) = \begin{cases} 1 + x, & x < 1, \\ 1 + x - 2(x - 1)^2, & x \geq 1. \end{cases}$$

Key properties (for $x \in [-2, 2]$): - **Intercept:** $\widehat{Y}(0) = 1$. - **At the knot $x = 1$:** $\widehat{Y}(1^-) = \widehat{Y}(1^+) = 2$ (continuous). - **Slope:** - $x < 1$: $\widehat{Y}'(x) = 1$. - $x \geq 1$: $\widehat{Y}'(x) = 1 - 4(x - 1) = 5 - 4x$. - Right-slope at $x = 1$ is 1 $\rightarrow$ continuously differentiable at the knot. - **Curvature for $x \geq 1$:** concave down (quadratic coefficient -2). - **Local maximum (for $x \geq 1$):**

$$\widehat{Y}'(x) = 0 \Rightarrow x^* = \frac{5}{4} = 1.25, \quad \widehat{Y}(1.25) = 1 + 1.25 - 2(0.25)^2 = 2.125.$$

- **Endpoint values:** $\widehat{Y}(-2) = -1$, $\widehat{Y}(2) = 1$.

Creating a sketch for this problem

```
[2]: # --- 1) Synthetic data ---
rng = np.random.default_rng(42)
n = 400
X = rng.uniform(-2, 2, size=n)

# basis functions
b1 = X
b2 = (X - 1.0)**2 * (X >= 1.0) # (X-1)^2 * I(X>=1)

# true coefficients
beta0, beta1, beta2 = 1.0, 1.0, -2.0

# noise
sigma = 0.4
eps = rng.normal(0.0, sigma, size=n)

# response
Y = beta0 + beta1*b1 + beta2*b2 + eps

# put into a DataFrame (handy for inspection)
df = pd.DataFrame({"Y": Y, "X": X, "b1": b1, "b2": b2})

# --- 2) Fit linear regression with the basis functions ---
X_design = sm.add_constant(df[["b1", "b2"]]) # adds intercept
model = sm.OLS(df["Y"], X_design).fit()
```

```

print(model.summary())

# --- 3) Compare estimated vs. true coefficients ---
est = model.params
comparison = pd.DataFrame({
    "parameter": ["Intercept", "beta1 (X)", "beta2 ((X-1)^2 I{X>=1})"],
    "true": [beta0, beta1, beta2],
    "estimated": [est["const"], est["b1"], est["b2"]],
})
print("\nEstimated vs. true coefficients:")
print(comparison.to_string(index=False))

```

OLS Regression Results

```

=====
Dep. Variable:          Y      R-squared:          0.857
Model:                OLS      Adj. R-squared:      0.857
Method:             Least Squares      F-statistic:      1193.
Date:                Sun, 12 Oct 2025      Prob (F-statistic):      1.41e-168
Time:                20:14:11      Log-Likelihood:      -205.14
No. Observations:      400      AIC:              416.3
Df Residuals:          397      BIC:              428.2
Df Model:                2
Covariance Type:      nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.9678	0.023	42.918	0.000	0.923	1.012
b1	1.0101	0.022	45.914	0.000	0.967	1.053
b2	-1.8335	0.136	-13.472	0.000	-2.101	-1.566

```

=====
Omnibus:                1.245      Durbin-Watson:          1.812
Prob(Omnibus):           0.537      Jarque-Bera (JB):        1.327
Skew:                   -0.098      Prob(JB):                0.515
Kurtosis:                2.797      Cond. No.:               7.73
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Estimated vs. true coefficients:

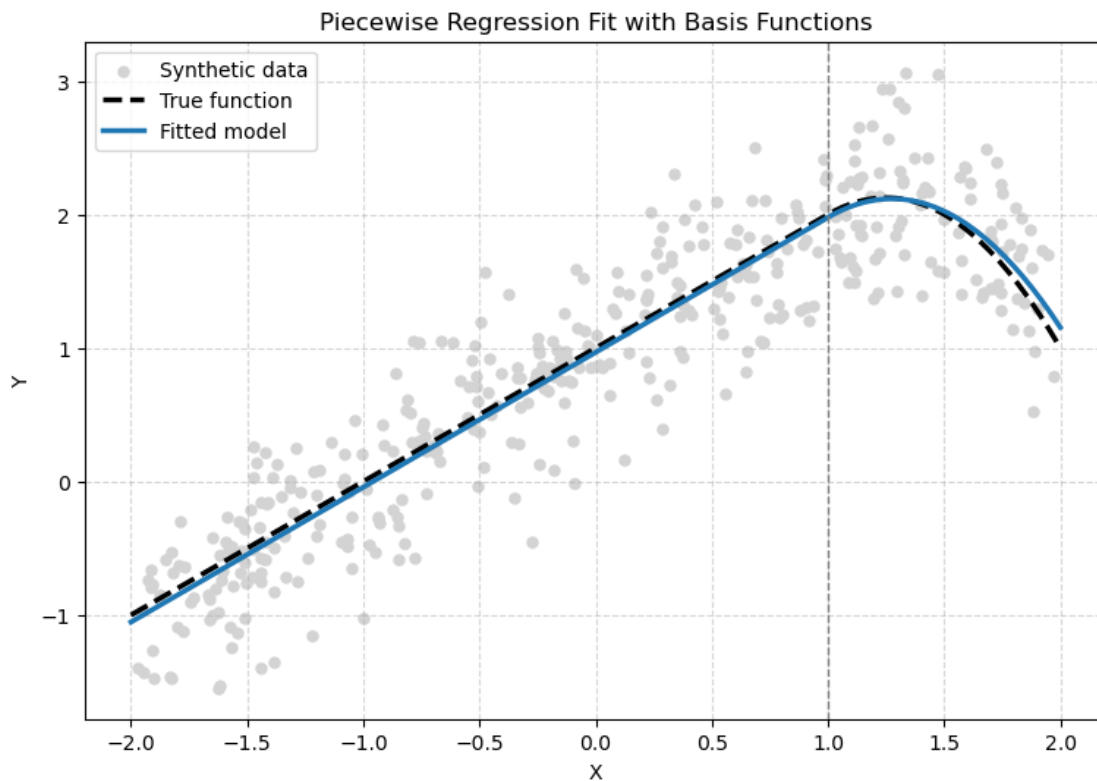
	parameter	true	estimated
	Intercept	1.0	0.967821
	beta1 (X)	1.0	1.010094
	beta2 ((X-1)^2 I{X>=1})	-2.0	-1.833530

```
[3]: # --- 4) Visualize results ---
xgrid = np.linspace(-2, 2, 200)
b1g = xgrid
b2g = (xgrid - 1)**2 * (xgrid >= 1)

# True and predicted curves
y_true = beta0 + beta1*b1g + beta2*b2g
y_pred = est["const"] + est["b1"]*b1g + est["b2"]*b2g

plt.figure(figsize=(9, 6))
plt.scatter(df["X"], df["Y"], color="lightgray", s=25, label="Synthetic data")
plt.plot(xgrid, y_true, "k--", lw=2.5, label="True function")
plt.plot(xgrid, y_pred, "tab:blue", lw=2.5, label="Fitted model")

plt.axvline(1, color="gray", ls="--", lw=1)
plt.title("Piecewise Regression Fit with Basis Functions")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



1.1.4 Interpretation

- The simulated data points (in gray) follow the expected nonlinear pattern: a straight line for $x < 1$ and a concave-down quadratic curve for $x \geq 1$.
- The dashed black line represents the **true function** $1 + x - 2(x - 1)^2 \mathbb{1}\{x \geq 1\}$.
- The solid blue line shows the **fitted regression model** using the same basis functions.

Observations: 1. The fitted curve aligns very closely with the true function, confirming that the linear model with these two basis functions can correctly capture the piecewise behavior.

2. The fit is smooth at the knot $x = 1$, as both the function and its first derivative are continuous there.

3. The estimated coefficients are close to the true values $(1, 1, -2)$, with minor differences due to sampling noise.

4. The model successfully adapts from a linear trend ($x < 1$) to a quadratic concave-down segment ($x \geq 1$), demonstrating the flexibility of basis function regression to represent nonlinear relationships.

1.2 Applied Exercises

1.2.1 (10) This question relates to the College data set.

1.2.2 (a) Split the data into a training set and a test set. Using out-of-state tuition as the response and the other variables as the predictors, perform forward stepwise selection on the training set in order to identify a satisfactory model that uses just a subset of the predictors.

```
[4]: College = pd.read_csv("../Data/College.csv")
```

```
[5]: College.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 777 entries, 0 to 776
```

```
Data columns (total 19 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	777 non-null	object
1	Private	777 non-null	object
2	Apps	777 non-null	int64
3	Accept	777 non-null	int64
4	Enroll	777 non-null	int64
5	Top10perc	777 non-null	int64
6	Top25perc	777 non-null	int64
7	F.Undergrad	777 non-null	int64
8	P.Undergrad	777 non-null	int64
9	Outstate	777 non-null	int64
10	Room.Board	777 non-null	int64
11	Books	777 non-null	int64
12	Personal	777 non-null	int64

```

13 PhD          777 non-null    int64
14 Terminal     777 non-null    int64
15 S.F.Ratio    777 non-null    float64
16 perc.alumni  777 non-null    int64
17 Expend       777 non-null    int64
18 Grad.Rate    777 non-null    int64
dtypes: float64(1), int64(16), object(2)
memory usage: 115.5+ KB

```

```
[6]: College.head()
```

```

[6]:
      Unnamed: 0 Private  Apps  Accept  Enroll  Top10perc  \
0  Abilene Christian University    Yes  1660    1232    721      23
1           Adelphi University    Yes  2186    1924    512      16
2           Adrian College    Yes  1428    1097    336      22
3       Agnes Scott College    Yes   417     349    137      60
4   Alaska Pacific University    Yes   193     146     55      16

      Top25perc  F.Undergrad  P.Undergrad  Outstate  Room.Board  Books  Personal  \
0           52         2885         537     7440      3300    450      2200
1           29         2683        1227    12280      6450    750      1500
2           50         1036          99    11250      3750    400      1165
3           89          510          63    12960      5450    450       875
4           44          249         869     7560      4120    800      1500

      PhD  Terminal  S.F.Ratio  perc.alumni  Expend  Grad.Rate
0    70         78        18.1         12    7041         60
1    29         30        12.2         16   10527         56
2    53         66        12.9         30    8735         54
3    92         97         7.7         37   19016         59
4    76         72        11.9          2   10922         15

```

We'll fit a logistic regression using **income** and **balance** to predict **default**. The response needs to be binary, so we map "Yes"→1, "No"→0.

```

[7]: # Get a copy of the DataFrame
df = College.copy()

# Drop non-informative ID column if present
if "Unnamed: 0" in df.columns:
    df = df.drop(columns=["Unnamed: 0"])

# Define target and features
y = df["Apps"]
X = df.drop(columns=["Apps"])

# Optional stratification by the `Private` flag (helps keep balance)
stratifier = X["Private"] if "Private" in X.columns else None

```

```

# Encode 'Private' as binary (0 = No, 1 = Yes)
X["Private"] = (X["Private"] == "Yes").astype(int)

# Split: 80% train / 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=stratifier
)

# Quick checks
print("Shapes:")
print("  X_train:", X_train.shape, "  y_train:", y_train.shape)
print("  X_test :", X_test.shape, "  y_test :", y_test.shape)

```

Shapes:

```

X_train: (621, 17)   y_train: (621,)
X_test  : (156, 17)   y_test  : (156,)

```

1.2.3 (b) Fit a GAM on the training data, using out-of-state tuition as the response and the features selected in the previous step as the predictors. Plot the results, and explain your findings.

```

[8]: smooth_vars = ["Room.Board", "PhD", "Terminal", "Expend", "Grad.Rate"]

# -----
# 1) Training bounds (for clipping)
# -----
lb = {v: float(X_train[v].min()) for v in smooth_vars}
ub = {v: float(X_train[v].max()) for v in smooth_vars}

def clip_to_train_bounds(df, vars_, lb_dict, ub_dict):
    out = df.copy()
    for v in vars_:
        out[v] = out[v].clip(lb_dict[v], ub_dict[v])
    return out

# -----
# 2) Build spline basis (no lower/upper kwargs)
# -----
dfs = [6]*len(smooth_vars)      # basis size per smoother (tune as needed)
deg = [3]*len(smooth_vars)      # cubic B-splines
bs = BSplines(X_train[smooth_vars], df=dfs, degree=deg)

# -----

```

```

# 3) Fit additive GAM (Gaussian identity)
# -----
gam = GLMGam(y_train, exog=X_train[["Private"]], smoother=bs)
res = gam.fit()
print(res.summary())

# -----
# 4) Predict on TEST with clipping
# -----
X_test_clip = clip_to_train_bounds(X_test, smooth_vars, lb, ub)

y_pred_test = res.predict(
    exog=X_test_clip[["Private"]],          # linear part
    exog_smooth=X_test_clip[smooth_vars],   # smooth part (CLIPPED)
    transform=True                          # identity link -> returns mean
)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2 = r2_score(y_test, y_pred_test)
print(f"Test RMSE: {rmse:,.1f} | Test R²: {r2:.3f}")

```

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          Apps      No. Observations:          621
Model:                  GLMGam    Df Residuals:              595
Model Family:           Gaussian  Df Model:                  25.00
Link Function:          Identity   Scale:                     9.2521e+06
Method:                 PIRLS     Log-Likelihood:            -5848.4
Date:                   Sun, 12 Oct 2025    Deviance:                  5.5050e+09
Time:                   20:14:11    Pearson chi2:              5.50e+09
No. Iterations:         3          Pseudo R-squ. (CS):        0.5165
Covariance Type:        nonrobust
=====

```

```

=====
=
              coef      std err          z      P>|z|      [0.025
0.975]
-----
-
Private      -5091.4042    361.062    -14.101    0.000    -5799.074
-4383.735
Room.Board_s0 1886.0552    2543.682     0.741    0.458    -3099.470
6871.580
Room.Board_s1 4044.6256    1778.164     2.275    0.023     559.487
7529.764
Room.Board_s2 2587.4109    2145.020     1.206    0.228    -1616.750
6791.572
Room.Board_s3 3973.2764    2091.694     1.900    0.057    -126.369
8072.922
=====

```

Room.Board_s4	7787.6745	2262.580	3.442	0.001	3353.099
1.22e+04					
PhD_s0	2299.5714	2960.187	0.777	0.437	-3502.289
8101.432					
PhD_s1	1003.0539	1735.345	0.578	0.563	-2398.161
4404.268					
PhD_s2	1919.1393	1837.918	1.044	0.296	-1683.115
5521.393					
PhD_s3	4713.6527	1829.361	2.577	0.010	1128.171
8299.135					
PhD_s4	261.7329	2071.745	0.126	0.899	-3798.812
4322.278					
Terminal_s0	1877.5560	2941.294	0.638	0.523	-3887.274
7642.386					
Terminal_s1	1023.8860	1964.170	0.521	0.602	-2825.817
4873.589					
Terminal_s2	515.6024	2037.787	0.253	0.800	-3478.387
4509.592					
Terminal_s3	1561.1528	1992.977	0.783	0.433	-2345.011
5467.317					
Terminal_s4	511.3760	2075.728	0.246	0.805	-3556.976
4579.728					
Expend_s0	-833.5792	1926.780	-0.433	0.665	-4609.999
2942.840					
Expend_s1	954.2142	1271.021	0.751	0.453	-1536.942
3445.370					
Expend_s2	2058.6743	2262.776	0.910	0.363	-2376.286
6493.635					
Expend_s3	1.118e+04	3507.989	3.187	0.001	4305.748
1.81e+04					
Expend_s4	3379.7419	3247.915	1.041	0.298	-2986.054
9745.538					
Grad.Rate_s0	-3172.4841	2961.849	-1.071	0.284	-8977.601
2632.632					
Grad.Rate_s1	-1245.7377	1822.551	-0.684	0.494	-4817.872
2326.396					
Grad.Rate_s2	225.6652	2254.708	0.100	0.920	-4193.482
4644.812					
Grad.Rate_s3	-555.8645	2227.045	-0.250	0.803	-4920.793
3809.064					
Grad.Rate_s4	5165.7183	3488.982	1.481	0.139	-1672.560
1.2e+04					

=====

=

Test RMSE: 2,565.9 | Test R^2 : 0.469

```

[9]: # -----
# 5) Plot smooth effects on a grid within TRAIN bounds
# -----
n_grid = 200
ref_private = float(X_train["Private"].mean()) # linear ref level

fig, axes = plt.subplots(2, 3, figsize=(13, 8))
axes = axes.ravel()

for i, col in enumerate(smooth_vars):
    # grid strictly within training support
    grid = np.linspace(lb[col], ub[col], n_grid)

    # other smooths fixed at train means; vary only 'col'
    Xi_grid = pd.DataFrame({v: np.full(n_grid, X_train[v].mean()) for v in
↪smooth_vars})
    Xi_grid[col] = grid

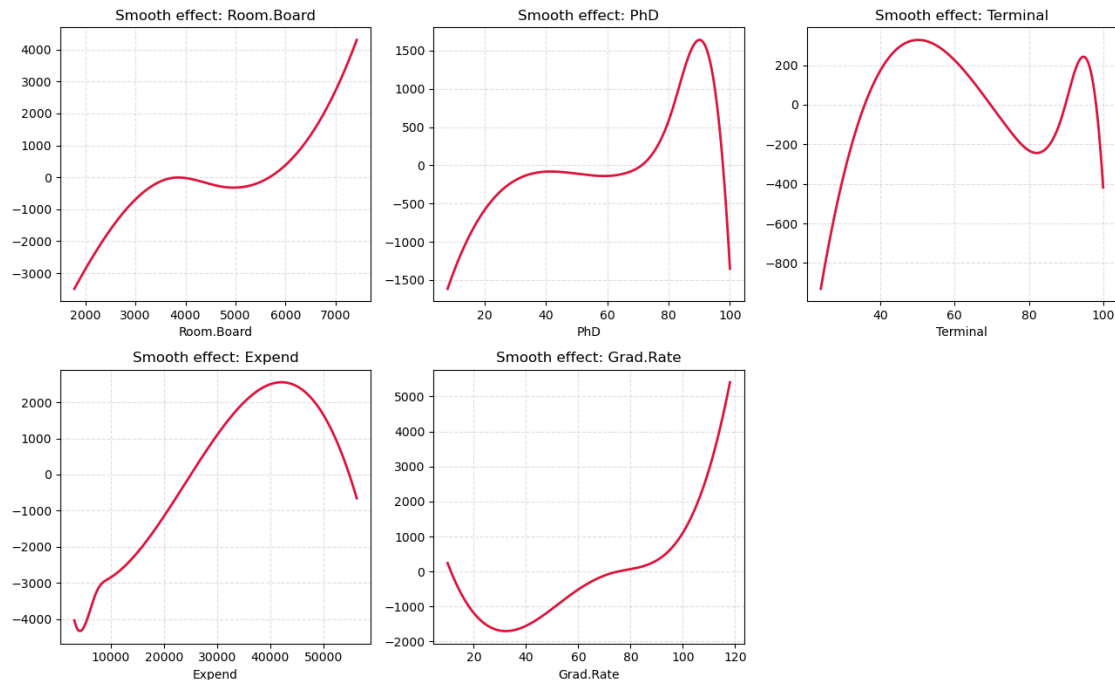
    # linear term fixed at its train mean
    exog_lin = pd.DataFrame({"Private": np.full(n_grid, ref_private)})

    # predict and center to show the shape
    yhat = res.predict(exog=exog_lin, exog_smooth=Xi_grid[smooth_vars],
↪transform=True)
    yhat_centered = yhat - yhat.mean()

    axes[i].plot(grid, yhat_centered, lw=2, color="crimson")
    axes[i].set_title(f"Smooth effect: {col}")
    axes[i].set_xlabel(col)
    axes[i].grid(True, linestyle="--", alpha=0.4)

axes[-1].axis("off")
plt.tight_layout()
plt.show()

```



Obs.: Each smooth-effect graph shows how the **predicted number of applications** changes with one variable, **holding all other predictors constant at their average values**; the vertical axis represents the **partial contribution** of that variable to the response, so upward trends indicate a **positive effect** (more applications), and downward trends a **negative effect**, relative to the mean level.

1.2.4 (c) Evaluate the model obtained on the test set, and explain the results obtained.

Comments (Response: Apps, n=621)

- **Test-set performance:**
Using the held-out data (30% of colleges), the model yields **RMSE 2,566** and **R² 0.47**.
- **Interpretation:**
The GAM explains roughly **47% of the variance** in the number of college applications for unseen institutions.
This indicates that the model generalizes reasonably well and captures the major nonlinear effects without overfitting.
- **Error magnitude:**
The average prediction error (~2.5k applications) is moderate compared to the scale of the response variable, showing that the fitted smooth relationships provide informative but not exhaustive predictive power.
- **Overall conclusion:**
The test results confirm that the GAM successfully generalizes the training relationships —

particularly the nonlinear effects of room/board, expenditures, and faculty qualifications — and provides a reliable, interpretable description of how institutional features shape student application volume.

1.2.5 (d) For which variables, if any, is there evidence of a non-linear relationship with the response?

The GAM reveals clear nonlinear effects for several predictors:

- **Room.Board**, **PhD**, **Expend**, and **Grad.Rate** show strong curvature in their smooth functions, indicating that changes in these variables affect applications in a non-constant way.
- **Terminal** displays a weaker, mild nonlinearity.

Overall, these patterns confirm that the relationship between institutional characteristics and application volume is **nonlinear**, justifying the use of a GAM over a simple linear model.